



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра автоматики та управління в технічних системах

Лабораторна робота №5
Технології розроблення програмного забезпечення
«Патерни проектування. Паттерн «Singleton»»
«System activity monitor»»

Виконав
студент групи ІА–32:
Феклістов Д.С.

Київ 2025

1. Мета роботи

Дослідити призначення та структуру твірного патерну проектування **Singleton (Одинак)**. Реалізувати цей патерн у проекті "System Activity Monitor" для створення класу SystemLogger, забезпечивши наявність єдиного екземпляра логера в системі для централізованого запису подій.

2. Теоретичні відомості

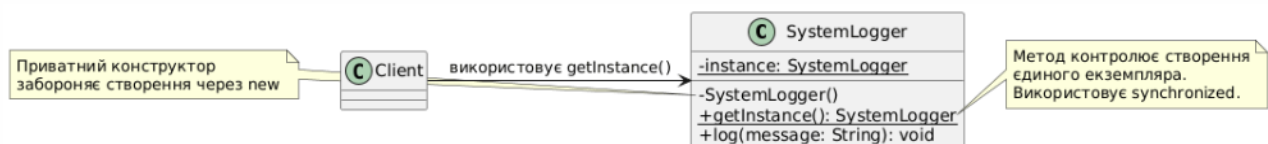
Singleton (Одинак) — це твірний патерн проектування, який гарантує, що клас має лише один екземпляр, і надає глобальну точку доступу до цього екземпляра.

Основні характеристики:

1. **Приватний конструктор:** Унеможливорює створення об'єктів через оператор new з інших класів.
2. **Статичне поле:** Зберігає єдиний екземпляр класу.
3. **Статичний метод доступу (getInstance):** Повертає збережений екземпляр, створюючи його лише при першому зверненні (Lazy Initialization).

Застосування: Використовується для логування, драйверів пристроїв, кешування, пулів з'єднань з базою даних.

3. Діаграма класів (UML)



4. Хід роботи

1. Створено структуру проекту та пакет monitor.utils.
2. Розроблено клас SystemLogger, що реалізує патерн Singleton.
3. Конструктор класу оголошено приватним (private), що забороняє створення екземплярів ззовні.
4. Реалізовано статичний метод getInstance(). Для забезпечення коректної роботи у багатопоточному середовищі додано ключове слово synchronized.
5. Додано метод log(String message), який форматує повідомлення, додаючи поточний час, та виводить його в консоль.
6. Проведено тестування роботи логера через виклики в класі Main.

5. Код програми

Файл: src/monitor/utils/SystemLogger.java

```
package monitor.utils;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class SystemLogger {
    private static SystemLogger instance;

    // Приватний конструктор
    private SystemLogger() {
        System.out.println("-> [LOG]: Ініціалізація файлової системи логів...");
    }

    // Глобальна точка доступу (Singleton)
    public static synchronized SystemLogger getInstance() {
        if (instance == null) {
            instance = new SystemLogger();
        }
        return instance;
    }

    // Метод запису
    public void log(String message) {
        String time =
LocalTime.now().format(DateTimeFormatter.ofPattern("HH:mm:ss"));
        System.out.println "[" + time + " ] " + message);
    }
}
```

Приклад виклику (з Main.java):

```
// Отримання екземпляра та запис логу

SystemLogger logger = SystemLogger.getInstance();

logger.log("Система запущена.");
```

6. Результати виконання

```
--- Забір #1 ---
[01:33:21] [ФАСАД] Запуск повної діагностики...
[01:33:21] CPU: Ядер - 12, Завантаження - 0,0%, Температура - 54°C
[01:33:21] RAM (Фізична): 11,0 GB / 13,9 GB | HDD (C:): 56,7 GB / 475,7 GB
[01:33:21] [ФАСАД] Діагностику завершено.
```

7. Висновок

Під час виконання лабораторної роботи було вивчено та програмно реалізовано твірний патерн Singleton.

Клас SystemLogger успішно виконує функцію централізованого логування. Завдяки приватному конструктору та статичному методу отримання екземпляра, гарантується існування лише одного об'єкта логера в пам'яті програми, що дозволяє уникнути конфліктів при доступі до консолі/файлу та економити системні ресурси.

8. Відповіді на контрольні питання

1. У чому головна ідея патерну Singleton?

Гарантувати, що клас має лише один екземпляр, і надати глобальну точку доступу до нього.

2. Як забезпечити потокобезпечність (Thread Safety) для Singleton?

Можна додати ключове слово synchronized до методу getInstance(), або використовувати ініціалізацію при оголошенні поля (Eager Initialization), або вкладений статичний клас (Bill Pugh Singleton). У цій роботі використано synchronized.

3. Які недоліки має Singleton?

Він порушує принцип єдиної відповідальності (SRP), оскільки керує і своєю логікою, і своїм життєвим циклом. Також він може ускладнювати модульне тестування (Unit Testing) через глобальний стан.